

Laboratorio Image and Video Analysis Giuliani

Gianluca Giuliani 7131139

August 25, 2025

1 Introduction

This project aims to evaluate the robustness of MASA tracker and create a heuristic to solve some errors in id assignments during detection. The full source code of the implementation can be found at the following GitHub repository: <https://github.com/gianlu9911/Progetto-IVA.git>

Within this repository, the main scripts of interest are:

- `featureMeanStd.py` – responsible for extracting intermediate deep features from the detector’s *neck*, computing statistical metrics (mean and standard deviation), and saving them for later analysis.
- `heuristic.py` – containing utility functions and heuristic-based decision rules to support id re-association.

Installation instructions are available on the GitHub repository page.

1.1 Prerequisites

- Python 3.11
- Installed dependencies: run `install_dependencies.bat` or `install_dependencies.sh` and download weights from the GitHub link

1.2 Troubles and Goal

MASA is a model that tracks and classifies elements inside a scene. What happens is that, the id assigned to a certain detection can change across frames. This is caused by collision of different tracked elements inside the scene or if the detected element exit from the scene. What normally happens is that the same person is defined with a new id after a collision with another element in the scene. The core idea is to exploit deep features extracted from the model to correct incorrect ID reassignments caused by occlusions or re-entries .

2 Script Description: `featureMeanStd.py`

2.1 Model Initialization

The script loads the MASA model using the provided configuration (`.py`) and checkpoint (`.pth`). A forward hook is registered on the model’s *neck* layer to capture intermediate feature maps during inference. The neck is chosen because it preserves spatial detail while embedding high-level semantic information.

It can be interesting using features extracted from a different point in the model but the required time to extract features is extremely relevant.

2.2 Frame-by-Frame Processing

The script opens the input video and processes frames sequentially. For each frame:

1. Runs MASA detection and tracking.
2. Retrieves the neck feature map from the forward hook.
3. For each detection above the score threshold:
 - Applies ROI Align to extract a fixed-size (7×7) feature representation.
 - Saves the resulting tensor to a `.numpy` file in the format:
`frame_<frame_id>_id_<track_id>_bbox_<x1>_<y1>_<x2>_<y2>.numpy`

The script produces feature files in `.numpy` format for each detection at each frame.

2.3 Usage

2.3.1 Command-line Execution

The script `featureMeanStd.py` can be executed as follows:

```
python my_scripts/featureMeanStd.py --detect --in_video videos/ins/video_03_5_person-comp.mp4
--out_video videos/outs/micc_2_out_unified.mp4 --out_tracks tracks/micc_2_tracks_unified.json
--unified --masa_config configs/masa-gdino/ masa_gdino_swinb_inference.py --masa_checkpoint
saved_models/masa_models/gdino_masa.pth
```

This will extract features from video number 3.

Argument explanation:

- `--detect` : Enables detection mode.
- `--in_video` : Path to the input video.
- `--out_video` : Path to save the processed video with overlays.
- `--out_tracks` : Path to save JSON tracking metadata.
- `--unified` : Runs MASA in unified detector + tracker mode.
- `--masa_config` : Path to MASA configuration file.
- `--masa_checkpoint` : Path to pretrained MASA model weights.

2.4 Future works

It would be interesting to experiment with features extracted from other layers of the model, but feature extraction time is a limiting factor.

3 Heuristic ID Association Pipeline

To improve identity association, A pipeline based on heuristics and incremental appearance model updates is developed. The main implementation is in `heuristics.py`. This tool is designed as a semi-automatic data refinement procedure: it combines automatic heuristics with user feedback to resolve ambiguous cases.

3.1 Data Parsing

Feature files are organized as: `frame_<frame_id>_id_<track_id>_bbox_<x1>_<y1>_<x2>_<y2>.numpy`. From each file, the following are extracted:

- **frame_id**: frame number
- **track_id**: tracking ID assigned by the detector
- **bbox**: associated bounding box

3.2 Appearance Model

Each identity is represented with an incremental Gaussian-like model updated via exponential moving averages:

$$\mu_{t+1} = (1 - \alpha)\mu_t + \alpha x, \quad \Sigma_{t+1} = (1 - \alpha)\Sigma_t + \alpha(x - \mu_{t+1})(x - \mu_{t+1})^T \quad (1)$$

where α is the learning rate and x the current frame feature.

Note that this update rule does not yield the true covariance but an exponentially weighted estimate of recent deviations. The model currently uses the Euclidean distance between a new feature x and the mean μ_t to assess similarity.

3.3 Dimensionality Reduction

To stabilize model updates, features are compressed via PCA. In the current implementation, PCA is fit once globally on all features before association:

$$X_{\text{PCA}} = \text{PCA}(X, \text{dim} = d)$$

where d is the final dimension chosen by the user. This reduces noise but is not adaptive: once fit, PCA does not update as new data is processed.

3.4 Why not Tsne?

t-SNE does not preserve distances and this leads to errors. Often the correct association it's not the first in the top-k or not listed at all because of the threshold. This makes it unsuitable for our case. Empirically, PCA with 16, 32, or 64 dimensions performs well."

3.5 Why not Mahalanobis?

Theoretically speaking Mahalanobis distance should consider even covariance and mean of the distributions. Empirically, Euclidean distance usually provides more stable associations.

3.6 Conflict Resolution

When multiple IDs appear in the same frame or one ID could correspond to multiple models, heuristic rules are applied:

- **Pause period:** after an association is made, the same ID is not reconsidered for a duration of `pause_sec × fps` frames, avoiding unstable rapid reassignments.
- **Skip IDs:** if an ID is marked as *skipped*, it is permanently excluded from further evaluation in all future frames.
- **Alias management:** new IDs can be explicitly associated with existing IDs, creating a mapping of aliases. This allows fragmented detections to be merged into a single consistent identity.
- **Candidate filtering:** candidates already present in the current frame are excluded. Only the top k candidates below a Euclidean threshold are considered.
- **User interaction:** in case of conflict, the user can select a candidate, accept the global best, skip the ID, or declare it as a new identity.

3.7 Auto-association and Statistics

If an ID repeatedly selects the same candidate for n consecutive frames, the association is automatically fixed. This mechanism acts as a temporal confirmation rule, reducing dependence on manual input and leveraging temporal consistency.

The pipeline also collects statistics:

- **Top1, Top2, Top3:** fraction of times the user-selected candidate corresponded to the first, second, or third best suggestion. These metrics do not measure ground-truth correctness, but act as a proxy for how well the ranking reflects user decisions.
- **Skipped IDs:** identifiers manually excluded from the association process.

3.8 Output

Final associations are saved in `id_associations.csv`, containing:

- `new_id`: detected ID
- `associated_to`: associated ID
- `frame`: frame in which the association was fixed

3.9 Limitations

While effective in practice, the pipeline has some limitations:

- PCA is computed once globally and does not adapt online.
- The covariance matrix is maintained but not used in a Mahalanobis distance; the system currently relies on Euclidean distance.
- Statistics measure agreement with user choices rather than ground-truth correctness, so they serve as a proxy rather than a formal evaluation metric.
- The pipeline is semi-automatic: ambiguous cases require human input, making it more suitable for *data cleaning and label refinement* rather than fully automated tracking.

3.10 Usage

The script `heuristics.py` can be executed as follows:

```
python my_scripts/heuristic.py --feature_dir saved_features_per_neck_2
```

This will elaborate features from video 2.

Argument explanation:

- `--feature_dir` : Select correct feature folder.
- `--dim` : Dimension after dimensionality reduction.
- `--euclidean_threshold` : Threshold of Euclidean distance; above this value candidates are not proposed.
- `--top_k` : Maximum number of candidates displayed for ID re-association.
- `--lock_after` : Blocks re-association after N consecutive equal choices.
- `--fps` : FPS of the video, used to compute the pause in frames.
- `--pause_sec` : Pause duration in seconds before evaluating the same ID again.

3.11 New vs. Returning IDs

The pipeline treats differently the case of a completely new identifier and the case of an identifier that reappears after being previously associated.

New IDs. When a track ID appears for the first time, it is compared with all existing appearance models. Candidates within the Euclidean threshold are proposed to the user, who can: (i) associate it to an existing identity (creating an alias), (ii) declare it as a new identity (Press Enter), or (iii) skip it entirely (press s).

3.12 About association:

User will have a top-k selection of possible to select using numbers ('1-5' for example for top-5). The user can press Enter (assign a new ID), s (skip), or x (force-assign to the nearest ID, ignoring filters).

3.13 Returning IDs.

Example. At frame x , track ID 26 was associated to identity 1. At frame $x + 1$, both track ID 35 (already associated to identity 1) and track ID 26 (temporarily drifting toward identity 3) are detected simultaneously. Since two aliases of the same identity cannot coexist in the same frame, the system pauses automatic updates and asks the user to resolve the conflict. The system triggers a conflict resolution step is triggered and the user must decide the correct association between the top-k suggestions of the system.

Code behavior in returning-ID conflicts. When a returning ID is detected, the code first checks whether it has already been assigned to an existing identity. If so, it verifies whether another alias of the same identity is currently active in the same frame. If no conflict is found, the feature of the returning ID is simply used to update the appearance model of its associated identity. If, however, a conflict is detected (e.g., ID 26 is known to belong to identity 1, but ID 35—also mapped to identity 1—is simultaneously present in the frame), the system pauses automatic updates and raises a conflict resolution dialog. At this point, the user is explicitly asked to decide among: (i) reassigning it to one of the near identities, or (ii) skipping it.

After a decision is made, the association table is updated accordingly and a temporal pause is enforced: the same ID will not be reconsidered for a fixed number of frames, computed as `pause_sec` $\times$ `fps`. During this period, the ID is ignored in order to prevent repeated queries and to stabilize the association process.

3.14 Results

We report the association statistics for Video 2 and Video 3. Figures 1 and 2 illustrate the distribution of user choices, auto-associations, and skipped identifiers collected during the heuristic pipeline execution.

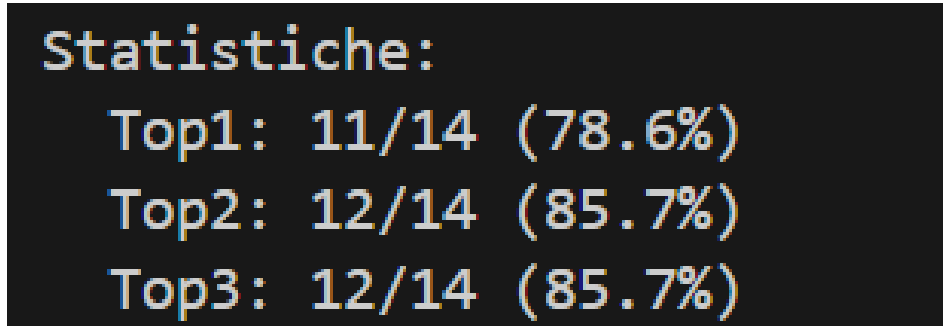


Figure 1: Association statistics for Video 2.

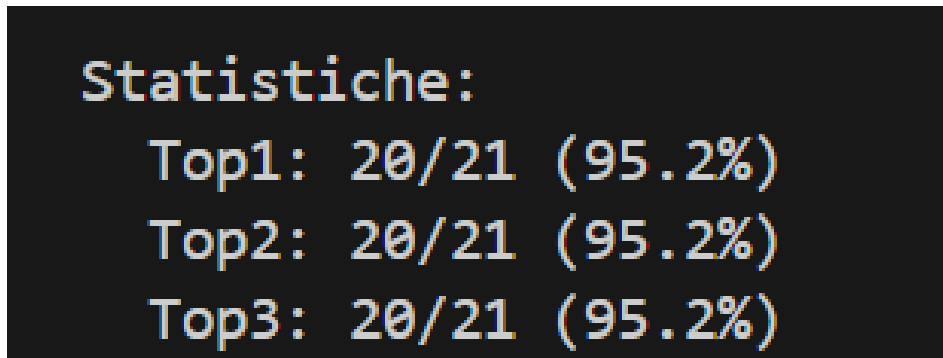


Figure 2: Association statistics for Video 3.